AGENCY OS: Gaze-Operated Agentic Computing with Fail-Safe Human Control

Stop Amplification in Multi-Surface Safety Controls and Approval Binding for Delegated Actions

Neil Gilani Ryan Cho Reyansh Bhakshi

October 2026

Abstract

People with severe motor impairments increasingly need to delegate everyday tasks—ordering food, paying bills, signing forms, calling for help—to software agents, yet they often cannot reach a keyboard, mouse, or physical kill switch. We present AGENCY OS, an open-source macOS system that combines webcam-only gaze input, an always-on-top accessibility overlay, a language-model agent constrained to a typed tool registry, a browser bridge, on-device memory, and an independent emergency channel (*Lifeline*). Our central design principle is that *the person's controls must be strictly more available than the agent's capabilities*: Help and Stop remain reachable with no permissions, no network, and no working agent. Building the system surfaced a failure mode we call *stop amplification*: when a safety command is broadcast to every surface and each surface echoes it, an idempotent stop becomes a feedback loop. In our integration build a single Stop produced roughly 12,600 stop messages. We show that the fix is structural—a single coalescing authority in the trusted process—and argue this is a general property of multi-surface human-override designs. We also describe *approval binding*, in which a person's consent is bound to a fingerprint of the exact live page element, so an approved action cannot be retargeted between consent and execution. We report engineering measurements from a working implementation (sub-second packaged startup, Stop reaching all subsystems within 500 ms) and state plainly what has not been evaluated: no study with disabled users has yet been run.

1 Introduction

Agentic software is moving from chat windows to systems that click, type, and pay on a person's behalf. For people living with ALS, spinal cord injury, or locked-in syndrome, such agents promise real independence, but they also concentrate risk: the person delegating the task is often the person least able to interrupt it. Conventional safeguards assume the user can press Escape, close a window, or pull a plug.

AGENCY OS starts from the opposite assumption. Its only required input is a commodity webcam; selection happens by dwelling gaze on large targets or by blinking. Every consequential action requires an explicit, deliberate approval, and two controls—**Help** and **Stop**—are designed to work when everything else has failed.

Contributions.

1. An architecture for gaze-operated agentic computing on a stock Mac, in which the agent, native input helper, browser bridge, and coordinator are separately supervised processes beneath a trusted Electron main process (§3).
2. The identification of *stop amplification*, a feedback failure in broadcast safety controls, with a measured instance and a structural remedy (§4).
3. *Approval binding*: consent attached to a fingerprint of the live target, re-verified at execution time by an independent component (§5).
4. An open implementation with honest status reporting: what is real, what is simulated, and what remains unvalidated (§7, §8).

2 Related Work

Gaze interaction has a long history in augmentative and alternative communication (AAC), from dedicated infrared trackers to dwell-based selection and the "Midas touch" problem of unintended activation. Webcam-based tracking using face landmark models has made gaze input cheaper but noisier, motivating large targets, adaptive calibration, and switch-scanning fallbacks. Separately, LLM-driven computer-use and web agents act through screenshots, accessibility trees, or DOM snapshots; their safety work has focused on prompt injection and action confirmation. Little prior work considers the intersection: an agent whose principal cannot easily intervene. Our work contributes system-level mechanisms for that setting rather than a new tracking or planning algorithm.

3 System Architecture

AGENCY OS is a monorepo of roughly 64k lines of TypeScript, Python, and Swift organised into independently tested packages:

- **Desktop host** (Electron): a shell window served over a custom `agency://` protocol with a strict content-security policy; an API proxy that injects the coordinator token so renderers never hold it; a typed `window.agency` bridge with per-window-role permission checks (e.g. the shell may not synthesise keystrokes or capture the screen).
- **Gaze engine**: MediaPipe face landmarks in a dedicated hidden window—the only window permitted camera access—producing a smoothed pointer, blink events, and a three-blink help signal. Calibration adaptively extends per-point collection up to a hard 8 s ceiling, which removed failures on slow hardware.
- **Overlay**: a transparent, always-on-top, click-through window per display showing a rail (Help, Stop, Home, Keyboard, Pause eyes) over any application.
- **Native helper** (Swift): Accessibility-API element discovery and synthetic input over JSON-RPC on stdio; refuses to type into secure (password) fields.
- **Agent**: an LLM planner in an isolated utility process; it can only invoke an allow-listed set of host methods.
- **Browser bridge**: a paired, sealed channel to a Chrome extension exposing page trees, clicks, typing, and navigation; page content is wrapped as untrusted data with injection heuristics.
- **Coordinator** (FastAPI/SQLite): plan validation, approvals, idempotent execution, verification, recovery, memory, and Lifeline escalation.

A supervisor restarts child processes with exponential backoff and health checks. Real-world side effects (payments, calls, messages, emergency dispatch) are *simulated by default*; the system never handles passwords, 2FA codes, or card numbers.

4 Stop Amplification

4.1 Observation

Stop must halt the agent, all synthetic input, and in-browser automation simultaneously, and it can be triggered from the overlay, the shell, a global hotkey, the browser extension, or the API. The natural implementation broadcasts: whichever surface receives Stop notifies every other surface, and each surface, upon being stopped, reports "stopped" so the others can update. During integration, the shell treated an incoming "stopped" notification as a reason to issue `stop.all` itself. One user Stop thus produced a cascade we measured at approximately **12,600** stop invocations before the processes saturated.

4.2 Why idempotence is not enough

Each individual stop was idempotent: stopping a stopped agent is harmless. The pathology lies in the *messages*, not the state. Let n surfaces each re-broadcast on receipt; with any echo path the message count grows without bound while the state stays fixed. The loop consumed the very IPC and CPU budget that a subsequent Help request would need—so a safety control degraded the availability of another safety control.

4.3 Remedy

We moved Stop to a single authority in the trusted main process. All surfaces send a *request*; main runs one stop at a time, coalescing concurrent requests into the in-flight operation, and fans out a single command to the agent, native helper, and bridge. Surfaces receive state, never a command to re-issue. Formally, the stop handler is a latch: requests arriving while *stopping* is set return the same pending result. After the change one user Stop produces exactly one fan-out, and our end-to-end tests assert that the fake agent and fake native helper both log the stop within 500 ms of each other.

Generalisation. Any human-override system with multiple entry points and mutual notification—kill switches across robot controllers, cancel buttons across microservices—risks the same amplification. We suggest the rule: *overrides are requests to one authority; notifications are state, never commands.*

5 Approval Binding

When the agent proposes a consequential web action (e.g. "Place order"), the person approves a description of it. Between approval and execution the page can change—through dynamic content, a malicious script, or a prompt-injected agent swapping element references. In AGENCY OS the browser extension computes a fingerprint of the live element and its consequential context; approval is bound to that fingerprint, and at click time the extension re-describes the element and refuses execution unless the fingerprints match. Crucially the check is performed by the bridge, not the agent, so a compromised planner cannot forge it. Document signing follows the same pattern with

a ceremony: the person must read the plain-language explanation and then perform a separate deliberate confirmation.

6 Lifeline

Help surfaces the shell to the front and opens an alert at the coordinator, escalating through contacts. The main process sends a heartbeat every 5 s independently of any renderer, and the help path retries when the coordinator is down. Coordinators can run on a separate Raspberry Pi so that a watchdog notices if the Mac itself dies. In the current build contacts' phones and dispatch are simulated.

7 Engineering Evaluation

All figures below were measured on a developer MacBook Pro (Apple M5 Pro, macOS 26.6) or in continuous integration; none involve human participants.

Measurement	Result
Coordinator sidecar median start (PyInstaller onedir, 47.7 MB)	397 ms
Packaged app ready (coordinator healthy)	0.9–1.2 s
Stop reaching agent and native helper	<500 ms apart
Stop messages per user Stop, before / after coalescing	≈12,600 / 1
Unsigned arm64 disk image	161 MB
Overlay touch targets	≥96 px

Desktop end-to-end tests launch the real Electron app and verify: one overlay per display with transparent, click-through pixels; that the shell is refused camera access and refused keystroke synthesis; Help opening an alert in the real coordinator; and Help still surfacing the shell when the coordinator is down. CI runs the full suite on Ubuntu and macOS, including Swift helper tests and the browser bridge's pairing tests (where we also fixed a race in which a channel could be marked paired after the socket had been replaced during an asynchronous seal).

8 Limitations

We have **not** conducted a study with people who have motor impairments; claims about usability, accuracy, and fatigue are untested. Webcam gaze accuracy was not benchmarked against a reference tracker. Payments, calls, messages, and emergency dispatch are simulated; the app is not yet code-signed or notarised. The stop-amplification count is a single measured incident, not a distribution. These are the next steps.

9 Conclusion

For agents serving people who cannot easily intervene, safety controls are the product. AGENCY OS demonstrates that a stock Mac and webcam can host a gaze-operated agent whose Help and Stop stay more available than its capabilities, and its development exposed stop amplification—a failure in which broadcast overrides undermine themselves—together with a simple structural fix. Code: https://github.com/NeilGilani/game.